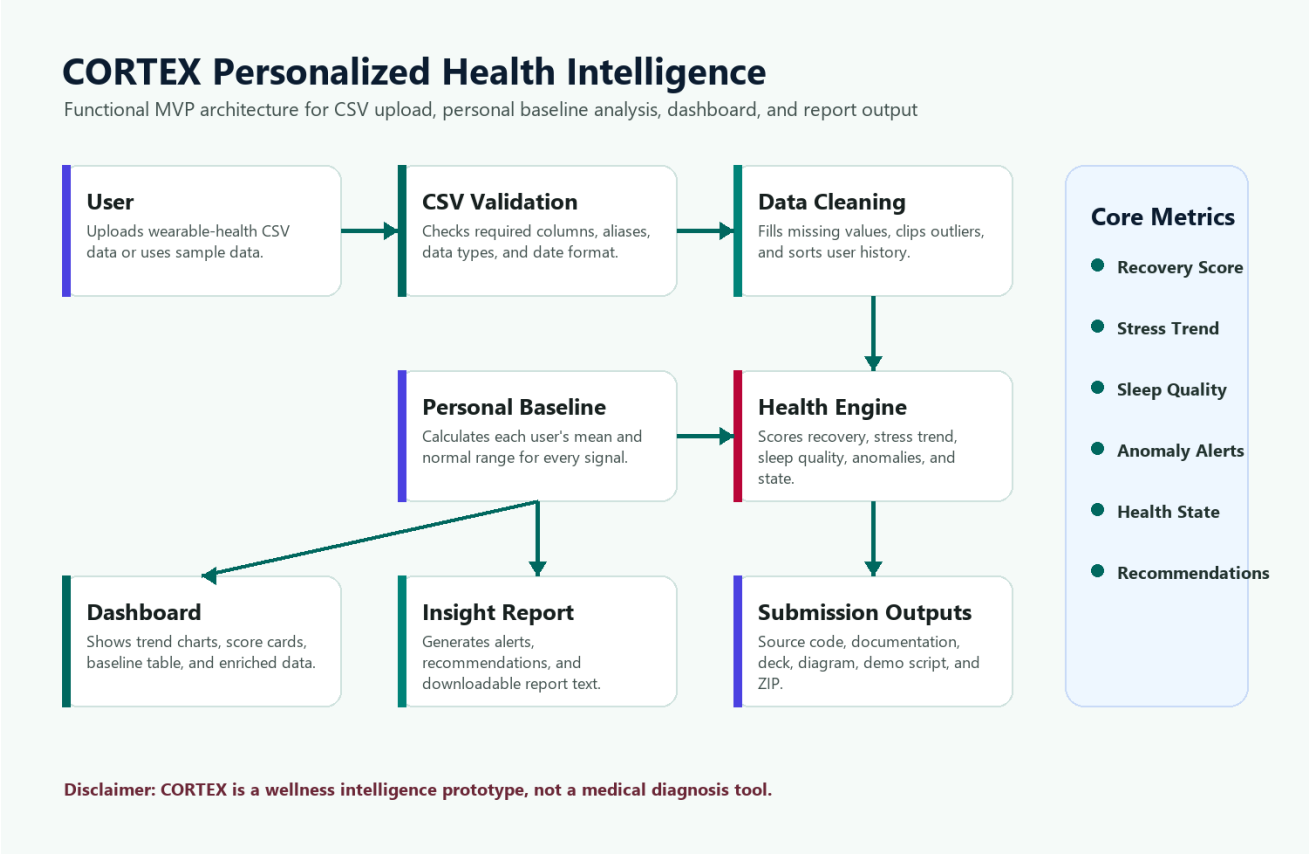
CORTEX Personalized Health Intelligence

Technical Documentation

This document explains the MVP problem, architecture, dataset, algorithm, dashboard, limitations, and future scope.



1. Problem Statement

Healthcare is still mostly reactive. Wearables collect heart rate, HRV, sleep, steps, oxygen saturation, and stress signals, but most dashboards show generic charts instead of understanding what is normal for one individual user. CORTEX solves this MVP problem by converting wearable CSV data into personalized recovery, stress, sleep, anomaly, and health-state insights.

2. Proposed Solution

CORTEX is a web-based health intelligence prototype. A user uploads a wearable-health CSV file. The system validates and cleans the data, creates a personal baseline from historical records, compares recent signals against that baseline, then generates a dashboard and final report.

The MVP focuses on practical wellness intelligence:

- Recovery Score from sleep, HRV, activity balance, SpO2, calmness, resting heart rate, and body temperature.
- Stress Trend from stress level, heart rate elevation, HRV drop, and sleep reduction.
- Sleep Quality from sleep duration and sleep score.
- Anomaly Alerts for abnormal HRV, resting heart rate, sleep, stress, SpO2, and temperature.
- Health State classification into Normal, Stressed, Fatigued, or Recovery Needed.

3. Dataset

The included demo dataset is `source_code/sample_health_data.csv`. It contains 1,500 wearable-health rows across 20 users, giving each user 75 days of history for baseline calibration and recent analysis. It uses these columns:

```
user_id,date,heart_rate,resting_heart_rate,hrv,sleep_hours,sleep_score,steps,active_minutes,calories_burne
```

Input features:

- heart_rate
- resting_heart_rate
- hrv
- sleep_hours
- sleep_score
- steps
- active_minutes
- calories_burned

- spo2
- respiratory_rate
- stress_level
- body_temperature

Training/evaluation targets:

- recovery_score, a 0-100 wellness recovery score.
- health_state, one of Normal, Stressed, Fatigued, or Recovery Needed.

4. System Architecture

The system is designed as a simple MVP pipeline:

1. CSV Upload
2. Data Validation
3. Data Cleaning and Normalization
4. Personal Baseline Creation
5. Feature Comparison and Health Scoring
6. Anomaly and Recommendation Engine
7. Dashboard, Report, and CSV Export

The architecture diagram is included as `documentation/Architecture_Diagram.png`.

5. Data Cleaning And Upload Validation

The cleaning layer is implemented in `source_code/utils.py`.

Main steps:

- Normalize column names and common aliases.
- Convert `date` into a real date/time field.
- Convert numeric health metrics into numeric values.
- Fill missing numeric values using user-level medians first.
- Fall back to dataset medians and safe defaults when a user has too little history.
- Clip values into realistic ranges to reduce extreme input errors.
- Sort each user's records by date.
- Normalize `stress_level` to a 0-100 scale. If an uploaded dataset uses a 1-10 stress scale, CORTEX multiplies it by 10 before scoring.

- Produce a visible validation workflow covering file received, required columns, date format, missing values, outliers, stress scale, baseline days, and analysis readiness.

This keeps the prototype stable when a CSV has blank values or slightly different column names.

6. Personal Baseline Model

The personal baseline is calculated in `source_code/model.py`.

For the selected user, CORTEX uses the earlier portion of the user's own history as the baseline. For every health feature, the system calculates:

- Personal mean
- Personal standard deviation

This makes CORTEX personalized. A heart rate, HRV, or sleep value is evaluated against the user's normal pattern rather than a generic population threshold.

7. Health Intelligence Algorithm

Recovery Score

The recovery score is computed from these components:

- Sleep component from sleep score and sleep hours.
- HRV component from HRV deviation against baseline.
- Activity balance from steps and active minutes versus baseline.
- SpO2 component from oxygen saturation.
- Calm component from inverse stress level.
- Penalties for elevated resting heart rate and abnormal temperature.

The final score is clipped to 0-100.

Stress Trend

CORTEX calculates a stress index using:

- Reported stress level.
- Heart rate elevation against baseline.
- HRV drop against baseline.
- Sleep reduction against baseline.

A short rolling average is compared with the longer recent average to classify the trend as Increasing, Stable, or Decreasing.

Health State Classification

CORTEX classifies the latest health state using interpretable rules and also calculates a confidence value:

- Recovery Needed: low recovery, high stress with HRV drop, or low SpO2.
- Stressed: high stress with elevated resting heart rate or HRV drop.
- Fatigued: sleep below baseline combined with HRV below baseline.
- Normal: stable signals and acceptable recovery.

The rule-based design is intentional for an MVP because it is explainable, debuggable, and easy to demonstrate.

8. Prototype Screens

The Streamlit app was rebuilt as a polished multi-page prototype using the Stitch design direction. It includes:

- Dashboard with recovery gauge, health-state card, HRV/stress/sleep/resting-HR metric cards, trend charts, and insights.
- Intelligence Hub with anomaly alerts, recommendations, baseline comparison, model metrics, and report downloads.
- Data Upload Validation with schema, missing value, outlier, stress-scale, and baseline checks.
- Profile & Baseline with calibration days, latest-vs-baseline deltas, and baseline tables.
- Patient Alerts with prototype patient-facing notification controls.
- Clinician Portal with multi-patient triage queue, critical recovery counts, notes, and PDF export.

9. Optional Machine Learning Training

The file `source_code/train_model.py` supports a time-based train/test workflow.

If scikit-learn is installed, it trains:

- A Random Forest Regressor for `recovery_score`.
- A Random Forest Classifier for `health_state`.

The split is time-aware: earlier dates are used for training and later dates are used for testing. If scikit-learn is not installed, the script evaluates the deterministic CORTEX rule engine instead.

Metrics are saved to:

```
source_code/model_artifacts/training_metrics.json
```

10. Dashboard

The Streamlit dashboard in `source_code/app.py` provides:

- CSV upload.
- Sample dataset loading.
- User profile selector.
- Recovery score, stress index, sleep quality, and health state cards.
- Heart rate, HRV, sleep, and stress charts.
- Baseline comparison table.
- Anomaly alerts.
- Recommendations.
- Downloadable PDF and text reports.
- Downloadable enriched CSV.

11. Tech Stack

- Python
- Streamlit
- Pandas
- NumPy
- Plotly
- Scikit-learn for optional model training
- ReportLab for PDF report export

This stack was chosen because it allows fast MVP delivery, readable logic, and a polished dashboard without building a separate backend.

12. Limitations

- The dataset is for prototype demonstration.
- CORTEX is not a medical diagnosis tool.
- The scoring engine uses interpretable rules, not a clinically validated model.
- No real wearable API integration is included in this MVP.
- No authentication or encrypted database is included.

- Clinician notes are stored only in the active Streamlit session for demo purposes.

13. Future Scope

- Connect Fitbit, Garmin, Apple Health, or Google Fit APIs.
- Add mobile app alerts.
- Store user data securely with encryption.
- Add doctor or coach review dashboards.
- Train larger time-series models on longer user histories.
- Add privacy-preserving analytics and consent management.

14. Run Instructions

```
cd source_code
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
streamlit run app.py
```

The app also includes deployment-ready files:

- `Dockerfile`
- `Procfile`
- `runtime.txt`
- `README_DEPLOY.md`

For quick verification:

```
python run_smoke_tests.py
python train_model.py
```